Mobile App Developer

Step 3: Learn Flutter or React Native

Concept & Learning Objective

Cross-platform frameworks aim to share substantial application code across platforms, but they do not remove platform differences. Flutter provides a Dart-based, widget-oriented UI toolkit; React Native uses JavaScript/TypeScript with React concepts and native platform capabilities.

Flutter's current learning guidance emphasizes Dart/Flutter fundamentals, app architecture and networking; its architecture guidance highlights separation of concerns, layered organization, single sources of truth, unidirectional data flow, dependency injection and testability. React Native similarly requires understanding components, state, navigation and native integration. ■cite■turn0search21■turn0search7■turn0search24■

Pick one framework for the main project. The important skill is not knowing every package; it is being able to structure a maintainable application.

Core Concepts — Detailed Breakdown

- Declarative UI: UI is derived from current state rather than manually manipulating every visual element.
- Components/widgets: reusable pieces with clear inputs and responsibilities.
- State: distinguish temporary UI state from shared application state; avoid putting everything into global state.
- Navigation: routes/screens, parameters, deep links and back-stack behavior.
- Networking: API client/service layer, serialization, loading/error states and cancellation.
- Architecture: separate presentation, state/domain logic and data access so features remain testable.
- Dependency management: use packages intentionally and verify compatibility.
- Platform integration: permissions, camera, storage, notifications and other native APIs may need platform-specific code.

What You Should Be Able To Do

- Build at least five reusable UI components.
- Navigate between three or more screens.
- Call a REST API and handle loading/error/empty states.
- Implement local state and one shared state flow.
- Explain where UI logic ends and data/business logic begins.

Hands-on Project / Practice

- Build a weather or expense tracker with multiple screens, API calls, local state and a clean service layer. Add tests for one important business rule.

Recommended References

- Flutter Learn — https://docs.flutter.dev/learn
- Flutter App Architecture — https://docs.flutter.dev/app-architecture
- Flutter State Management — https://docs.flutter.dev/data-and-backend/state-mgmt/intro
- React Native Documentation — https://reactnative.dev/docs/getting-started